==========

==========

CHAPTER 22 — DEVICE MAPPING & I/O VIRTUALIZATION

==========

==========

A maximum-detail, RTOS/QNX-style breakdown of ARMv8 device memory mapping, MMIO rules, PCIe BAR handling, interrupt controller virtualization, DMA/IOMMU integration, and safe I/O partitioning.

This chapter focuses on **how real systems safely map devices**, enforce ordering, and virtualize I/O for guests and partitions.

****

****

22.1 — Introduction to Device Mapping

****

****

Device mapping is one of the most dangerous and subtle parts of MMU design.

Incorrect device mapping causes:

- device malfunction
- bus lockups
- corrupted registers
- DMA corruption
- unpredictable timing
- security vulnerabilities

ARMv8 requires **strict rules** for device memory:

- non-cacheable
- non-speculative
- non-prefetchable
- strongly ordered
- 4 KB pages only
- Device-nGnRnE

This chapter explains how to map devices safely and how hypervisors virtualize them.

****

****

22.2 — Device Memory Types

****

****

Device memory must use **MAIR Device encodings**:

Device-nGnRnE (0x00)

- non-Gathering
- non-Reordering

- non-Early-Ack
- strongest ordering
- safest for MMIO

Device-nGnRE (0x04)

- early write acknowledgment allowed
- used for some high-speed devices

Device-nGRE (0x08)

- limited reordering allowed
- rarely used

Rules

- MMIO must be Device-nGnRnE
- DMA control registers must be Device-nGnRnE
- PCIe BARs must be Device-nGnRnE

****

****

22.3 — Why Device Memory Must Be Non-Cacheable

****

****

Caching device memory causes:

- stale register reads

- reordered writes
- lost writes
- corrupted device state
- undefined behavior

Device memory must be:

- non-cacheable
- non-speculative
- non-prefetchable

****

****

22.4 — Device Memory Mapping Rules

****

****

Rule 1 — Use 4 KB pages only

Blocks (2 MB / 1 GB) are forbidden for MMIO.

Rule 2 — Use Device-nGnRnE

Strongest ordering.

Rule 3 — Use Non-Shareable

Device memory must not be coherent.

Rule 4 — Use AP=RW

Most MMIO registers require write access.

Rule 5 — Use PXN/UXN

Device memory must never be executable.

Rule 6 — Use DSB SY after writes

Ensures ordering.

Rule 7 — Use DSB SY before reads

Ensures visibility.

****

****

22.5 — MMIO Access Ordering

****

****

Write Ordering

```
STR x0, [MMIO]
DSB SY
```

Read Ordering

```
DSB SY
LDR x0, [MMIO]
```

Why?

Devices require strict ordering to function correctly.

****

****

22.6 — PCIe BAR Mapping

****

****

PCIe Base Address Registers (BARs) expose device memory.

Rules

- map BARs as Device-nGnRnE
- use 4 KB pages
- never use cacheable attributes
- never use blocks
- enforce ordering with DSB

Hypervisor Virtualization

Hypervisor traps guest MMIO accesses and emulates device behavior.

****

****

22.7 — GIC (Interrupt Controller) Mapping

****

****

ARM GIC components:

- **GICD** — Distributor
- **GICC** — CPU Interface
- **GICR** — Redistributor
- **GITS** — ITS (Interrupt Translation Service)

Mapping Rules

- Device-nGnRnE
- Non-Shareable
- 4 KB pages
- PXN/UXN
- AP=RW

Virtualization

Hypervisor virtualizes:

- interrupt routing
- interrupt injection
- interrupt masking
- virtual CPU interfaces

****

****

22.8 — Timer Mapping

****

****

ARM timers:

- CNTV — virtual timer
- CNTP — physical timer
- CNTPS — secure timer

Mapping Rules

- Device-nGnRnE
- Non-Shareable
- 4 KB pages

Virtualization

Hypervisor traps guest timer accesses and emulates timer behavior.

****

****

22.9 — UART, SPI, I2C, GPIO Mapping

****

****

All low-speed peripherals must be mapped:

- Device-nGnRnE
- Non-Shareable
- 4 KB pages
- AP=RW
- PXN/UXN

Why?

These devices require strict ordering and non-cacheable access.

****

****

22.10 — DMA Engines

****

****

DMA engines operate outside CPU caches.

DMA Control Registers

- must be Device-nGnRnE
- must be Non-Shareable
- must use 4 KB pages

DMA Buffers

- must be Normal WB or Normal NC
- must be Inner Shareable
- must be aligned

Cache Maintenance

Before DMA read:

```
DC CVAC, buffer
DSB SY
```

After DMA write:

```
DC CIVAC, buffer
DSB SY
```

****

****

22.11 — IOMMU Integration

****

****

IOMMU performs:

```
Device DMA → IOVA → PA
```

Benefits

- DMA isolation
- device virtualization
- SR-IOV support
- protection against DMA attacks

Mapping Rules

- IOVA → PA mappings use Normal WB
- MMIO uses Device-nGnRnE
- DMA buffers use Normal WB or NC

****

****

22.12 — Device Virtualization Techniques

****

****

Technique 1 — Trap-and-Emulate

Hypervisor intercepts MMIO accesses.

Technique 2 — Direct Assignment

Device passed directly to VM via IOMMU.

Technique 3 — Para-Virtualized Drivers

Guest uses hypercalls instead of MMIO.

Technique 4 — SR-IOV

Hardware exposes virtual functions.

****

****

22.13 — Virtualizing MMIO

****

****

Hypervisor intercepts:

- MMIO reads
- MMIO writes

Hypervisor emulates:

- device registers
- device state
- interrupts
- DMA behavior

Performance

Trap-and-emulate is expensive → para-virtualization preferred.

****

****

22.14 — Virtualizing Interrupts

****

****

Hypervisor virtualizes:

- interrupt routing
- interrupt masking
- interrupt injection
- virtual CPU interfaces

GIC Virtualization

- GICv2 → VGIC
- GICv3 → GICR virtualization
- GICv4 → direct injection

****

****

22.15 — Virtualizing Timers

****

****

Hypervisor virtualizes:

- CNTV registers
- timer interrupts
- timer frequency
- timer offsets

Why?

Guest must see consistent virtual time.

****

****

22.16 — Virtualizing PCIe

****

****

Hypervisor virtualizes:

- PCIe configuration space
- BARs
- MSI/MSI-X interrupts
- DMA via IOMMU

SR-IOV

Hardware exposes multiple virtual functions.

****

****

22.17 — Device Partitioning

****

****

Devices must be partitioned:

Safety Partition

- safety-critical MMIO
- deterministic timing
- isolated DMA

Non-Safety Partition

- general-purpose devices
- shared DMA

Hypervisor Partition

- controls all MMIO

- enforces isolation

****

****

22.18 — Common Device Mapping Mistakes

****

****

Mistake 1 — Mapping MMIO as Normal WB

Catastrophic device malfunction.

Mistake 2 — Using blocks for MMIO

Breaks ordering.

Mistake 3 — Forgetting DSB

Writes reorder.

Mistake 4 — Forgetting PXN/UXN

Device memory becomes executable.

Mistake 5 — Incorrect DMA attributes

Data corruption.

****

****

22.19 — Full Device Mapping Example

****

****

MMIO Mapping

```
Descriptor:
    AttrIndx = 2 (Device-nGnRnE)
    SH = 00 (Non-Shareable)
    AP = RW
    PXN = 1
    UXN = 1
```

Access Sequence

```
STR x0, [MMIO]
DSB SY
LDR x1, [MMIO]
```

DMA Buffer Mapping

```
Descriptor:
    AttrIndx = 0 (Normal WB)
    SH = 11 (Inner Shareable)
```

****

****

22.20 — Chapter Summary

****

****

This chapter covered:

- device memory types
- MMIO mapping rules
- PCIe BAR mapping
- GIC virtualization
- timer virtualization
- UART/SPI/I2C/GPIO mapping
- DMA engines
- IOMMU integration
- device virtualization techniques
- MMIO trap-and-emulate
- interrupt virtualization
- PCIe virtualization
- device partitioning
- common mistakes
- full examples